

Auth Guard in Angular (20+): Conceptual Overview

In Angular (including 20+), an **Auth Guard** is part of the Angular Router's *navigation lifecycle*. It acts as a gatekeeper that determines whether a route can be activated, deactivated, loaded, or matched—based on some condition (typically authentication/authorization).

Think of it as **middleware for routing**: before Angular renders a component for a route, it asks the guard, “*Is this navigation allowed?*”

Why Auth Guards Exist

In real-world apps:

- Some routes are **public** → /login, /register
- Some routes are **protected** → /dashboard, /orders

Without guards, users could manually enter URLs and bypass UI-level restrictions.

Auth Guards enforce:

- **Authentication** → Is the user logged in?
 - **Authorization** → Does the user have permission (role-based)?
-

Types of Route Guards in Angular

Angular Router provides multiple guard interfaces (or functional equivalents in newer versions):

Guard Type	Purpose
CanActivate	Controls access to a route
CanActivateChild	Controls access to child routes
CanDeactivate	Prevents leaving a route (e.g., unsaved form)
CanLoad	Prevents lazy-loaded modules from loading
CanMatch (<i>modern replacement for CanLoad</i>)	Decides if a route should even match

 In Angular 15+, **functional guards** are preferred over class-based guards.

⚙️ Core Idea (How It Works)

When a user tries to navigate:

User → Router → Guard → Decision → Route or Block

The guard returns:

- true → allow navigation
- false → block navigation
- UrlTree → redirect to another route (e.g., /login)

✳️ Functional Auth Guard (Angular 20+ style)

Angular 20+ encourages **standalone + functional APIs**.

Example: Auth Guard

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  } else {
    return router.createUrlTree(['/login']);
  }
};
```

Auth Service (Simple Logic)

```
@Injectable({ providedIn: 'root' })  
  
export class AuthService {  
  
  isLoggedIn(): boolean {  
    return !!localStorage.getItem('token');  
  }  
}
```

Applying Guard to Routes

```
import { Routes } from '@angular/router';  
  
import { DashboardComponent } from './dashboard.component';  
  
import { authGuard } from './auth.guard';  
  
export const routes: Routes = [  
  { path: 'login', component: LoginComponent },  
  {  
    path: 'dashboard',  
    component: DashboardComponent,  
    canActivate: [authGuard]  
  }  
];
```

Redirect Flow

If user is NOT logged in:

/dashboard → authGuard → false → redirect → /login

If logged in:

/dashboard → authGuard → true → component loads

When to Use Which Guard

- Use CanActivate → protect main routes
 - Use CanActivateChild → protect nested routes
 - Use CanMatch → control lazy loading access
 - Use CanDeactivate → warn before leaving (forms)
-

Real-World Example

Imagine an e-commerce app:

- /products → public
- /cart → requires login
- /admin → requires admin role

Auth Guard checks:

```
if (user.role === 'admin') return true;
```
